OOP in C++

Inheritance

Instructor: Dinesh Maharjan

2022

Inheritance

- Creating new classes from existing classes is called inheritance.
- New class is called derived class, child class.
- Existing class is called base class, parent class.
- The child class will contain all the methods and attributes of parent class.
- However, private members of parent class will not be accessible from child class.
- Child class can have new attributes and methods.
- Child class can override the attributes and methods in the parent class.

Why Inheritance

- Instead of creating new class from scratch, we create new class by inheriting the existing class.
- This provides the opportunity to reuse the existing code in many situations.
- This eases to create new classes.
- We can create as many as derived class.
- Save time and effort.
- Increases program reliability.

Design Without Inheritance

- One class per asset type:

Stock
symbol_ total_shares_ total_cost_ current_price_
GetMarketValue() GetProfit() GetCost()

DividendStock
symbol_ total_shares_ total_cost_ current_price_ dividends_
GetMarketValue() GetProfit() GetCost()

Cash
amount_
GetMarketValue()

  - Redundant!
  - Cannot treat multiple investments together
    - *e.g.* can't have an array or `vector` of different assets

Inheritance

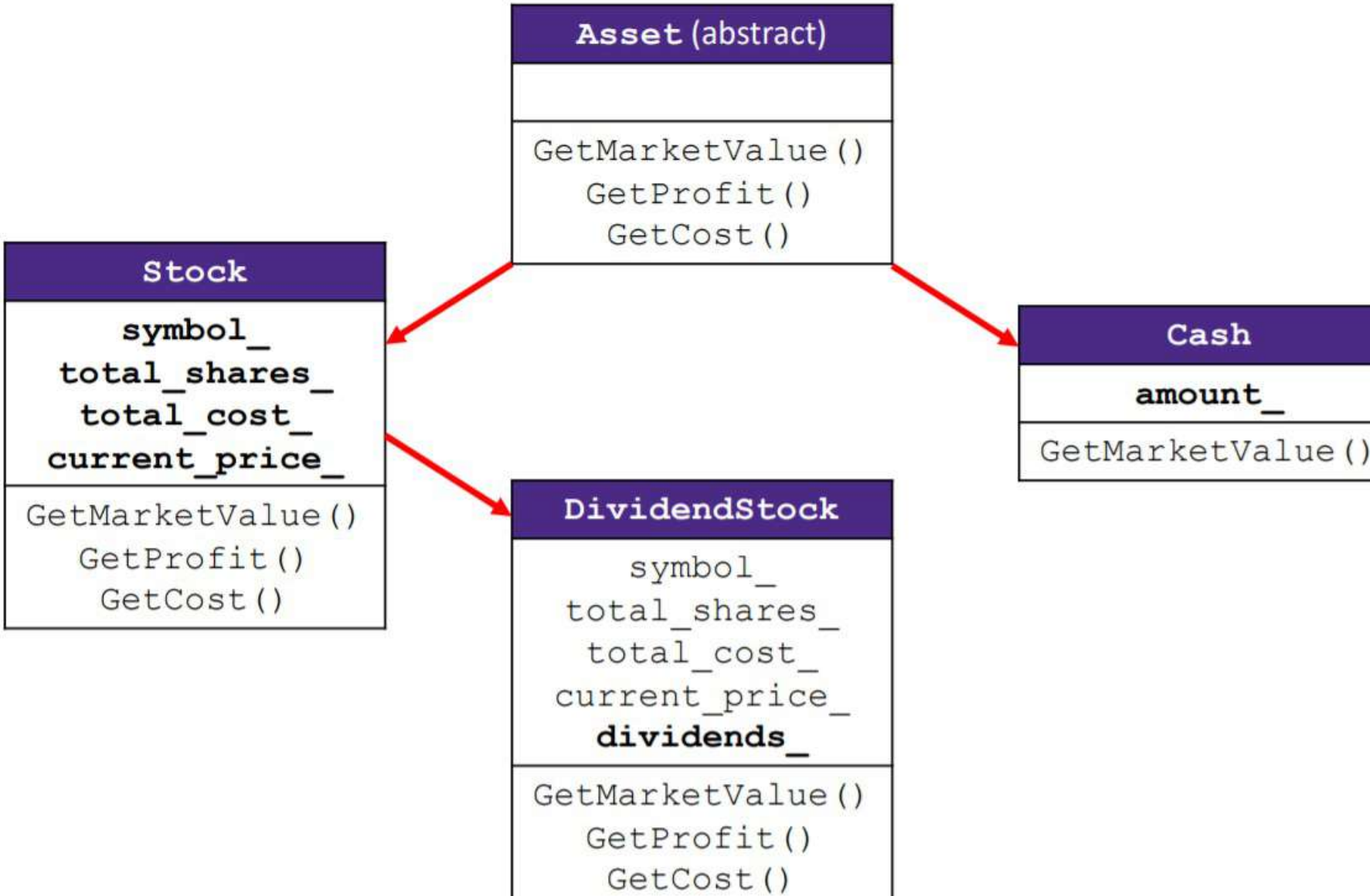
- A parent-child "is-a" relationship between classes
  - A child (derived class) extends a parent (base class)
- Benefits:
  - Code reuse
    - Children can automatically inherit code from parents
  - Polymorphism
    - Ability to redefine existing behavior but preserve the interface
    - Children can override the behavior of the parent
    - Others can make calls on objects without knowing which part of the inheritance tree it is in
  - Extensibility
    - Children can add behavior

Terminology

Java	C++
Superclass	Base Class
Subclass	Derived Class

- Mean the same things. You'll hear both.

Design With Inheritance

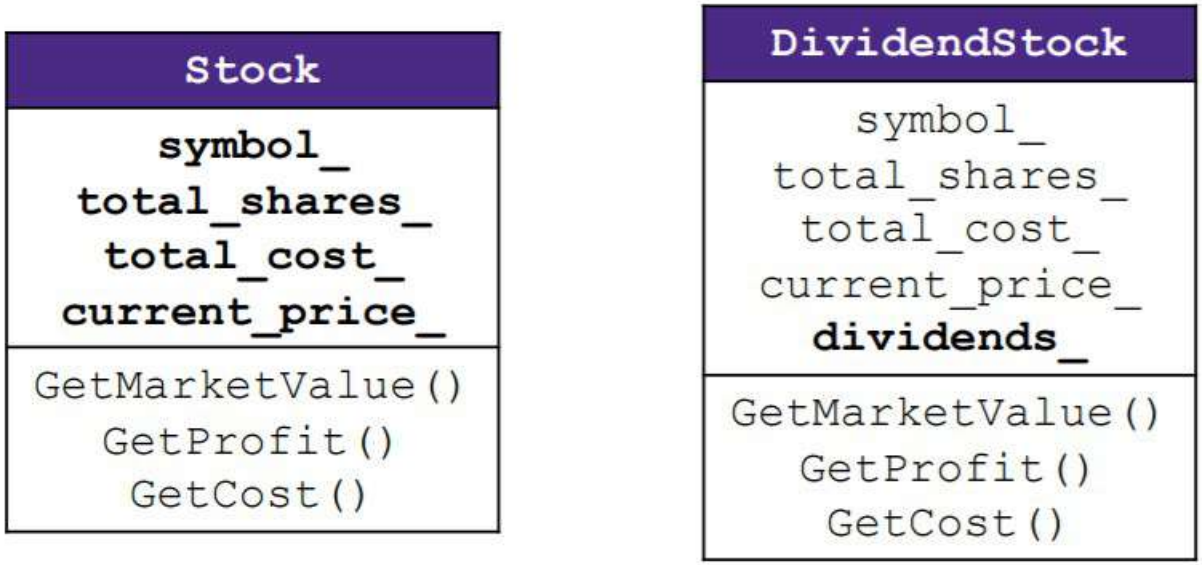


Like Java: Access Modifiers

- `public`: visible to all other classes
- `protected`: visible to current class and its *derived* classes
- `private`: visible only to the current class

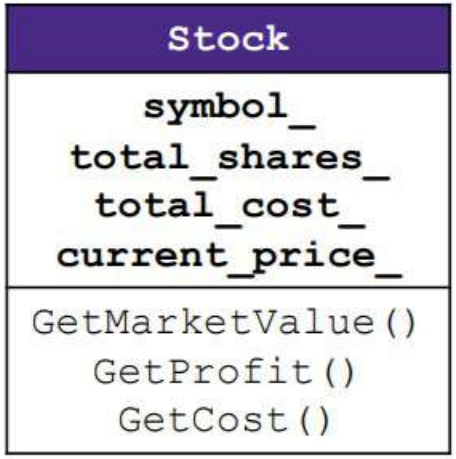
- Use `protected` for class members only when
  - Class is designed to be extended by subclasses
  - Subclasses must have access but clients should not be allowe

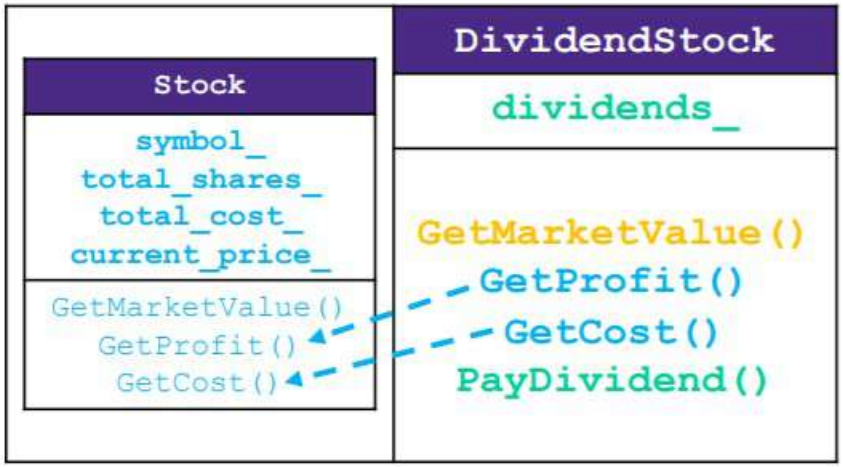
Back to Stocks



BASE DERIVED

Back to Stocks





- A derived class:
  - **Inherits** the behavior and state (specification) of the base class
  - **Overrides** some of the base class' member functions (opt.)
  - **Extends** the base class with new member functions, variables (opt.)

Derived Classes

- sometimes, want to express commonality between classes
- want to create new class from existing class by adding new members or replacing (i.e., hiding/overriding) existing members
- can be achieved through language feature known as ***inheritance***
- generate new class with all members of already existing class, excluding special member functions (i.e., constructors, assignment operators, and destructor)
- new class called **derived class** and original class called **base class**
- derived class said to **inherit** from base class
- can add new members (not in base class) to derived class
- can hide or override member functions from base class with new version
- syntax for specifying derived class:

  `class` *derived_class* : *base_class_specifiers*

- *derived_class* is name of derived class; *base_class_specifiers* provide base-class information

Person Class

```
1  #include <string>
2
3  class Person {
4  public:
5      Person(const std::string& family_name,
6        const std::string& given_name) :
7        family_name_(family_name), given_name_(given_name) {}
8      std::string family_name() const {return family_name_;}
9      std::string given_name() const {return given_name_;}
10     std::string full_name() const
11       {return family_name_ + ", " + given_name_;}
12     // ...
13 private:
14     std::string family_name_;
15     std::string given_name_;
16 };
```

Student Class Without Inheritance

```
1  #include <string>
2
3  class Student {
4  public:
5      Student(const std::string& family_name,
6        const std::string& given_name) :
7        family_name_(family_name), given_name_(given_name) {}
8        // NEW
9      std::string family_name() const {return family_name_;}
10     std::string given_name() const {return given_name_;}
11     std::string full_name() const
12       {return family_name_ + ", " + given_name_;}
13     std::string student_id() {return student_id_;} // NEW
14 private:
15     std::string family_name_;
16     std::string given_name_;
17     std::string student_id_; // NEW
18 };
```

Student Class With Inheritance

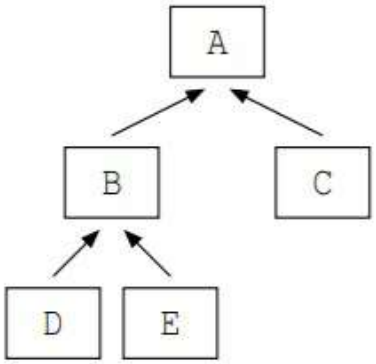
```
1  // include definition of Person class here
2
3  class Student : public Person {
4  public:
5      Student(const std::string& family_name,
6        const std::string& given_name,
7        const std::string& student_id) :
8        Person(family_name, given_name),
9        student_id_(student_id) {}
10     std::string student_id() {return student_id_;}
11 private:
12     std::string student_id_;
13 };
```

Class Hierarchies

- inheritance relationships between classes form what is called **class hierarchy**
- often class hierarchy represented by directed (acyclic) graph, where nodes correspond to classes and edges correspond to inheritance relationships
- class definitions:

```
class A { /* ... */ };
class B : public A { /* ... */ };
class C : public A { /* ... */ };
class D : public B { /* ... */ };
class E : public B { /* ... */ };
```

- inheritance diagram:

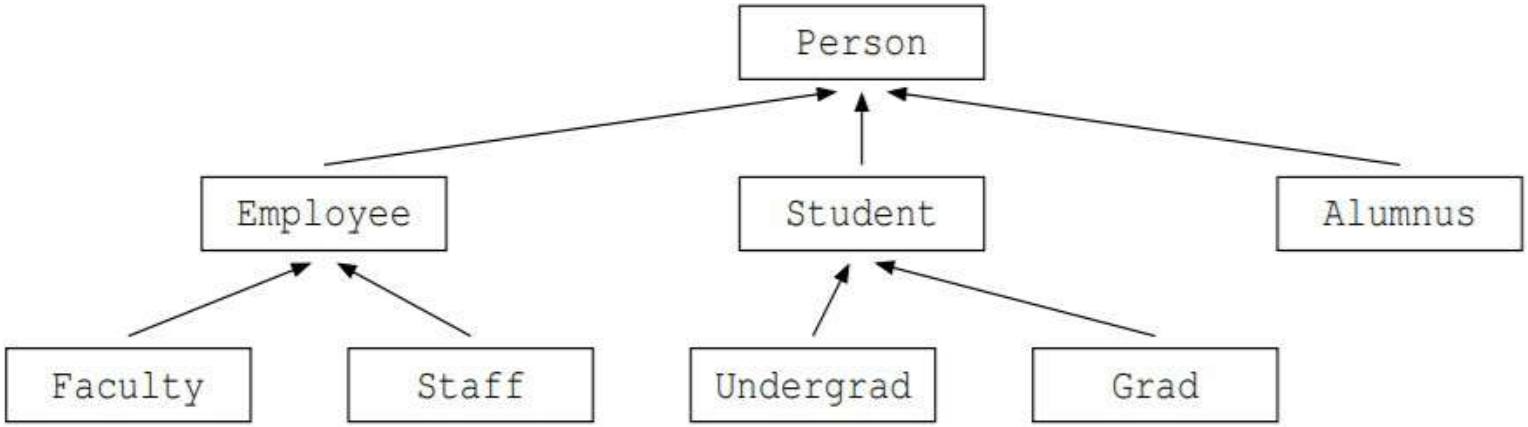


Class Hierarchy Example

- class definitions:

```
class Person { /* ... */ };
class Employee : public Person { /* ... */ };
class Student : public Person { /* ... */ };
class Alumnus : public Person { /* ... */ };
class Faculty : public Employee { /* ... */ };
class Staff : public Employee { /* ... */ };
class Grad : public Student { /* ... */ };
class Undergrad : public Student { /* ... */ };
```

- inheritance diagram:



- each of `Employee`, `Student`, and `Alumnus` is a `Person`; each of `Faculty` and `Staff` is an `Employee`; each of `Undergrad` and `Grad` is a `Student`

Member Access Specifiers: `protected`

- earlier, introduced **`public`** and **`private`** access specifiers for class members
- in context of inheritance, another access specifier becomes relevant, namely, **`protected`**
- member declared in protected section of class can only be accessed by
  - member functions and friends of that class; and
  - by member functions and friends of ***derived classes***
- protected members used to provide developers of derived classes access to some inner workings of base class without exposing such inner workings to everyone
- usually, bad idea to use protected access for data members (for similar reasons that using public access for data members is usually bad)
- protected access usually employed for function members

Types of Inheritance

- three types of inheritance with respect to access protection: public, protected, and private
- these three types of inheritance differ in terms of accessibility, in derived class, of members inherited from base class
- private parts of base class are always inaccessible in derived class, regardless of whether public, protected, or private inheritance used
- if this were not case, all access protection could simply be bypassed by using inheritance
- access specifiers for members accessible in derived class chosen as follows:

Access Specifier in Base Class	Access Specifier in Derived Class		
	Public Inheritance	Protected Inheritance	Private Inheritance
public	public	protected	private
protected	protected	protected	private

Types of Inheritance (Continued)

- for struct, defaults to public inheritance
- for class, defaults to private inheritance
- public and protected/private inheritance have different use cases, as we will see later

Inheritance and Member Access Example

```
class Base {
public:
    void f();
protected:
    void g();
private:
    int x;
};

class Derived_1 : public Base {
    // f is public
    // g is protected
    // x is not accessible from Derived_1
};

class Derived_2 : protected Base {
    // f is protected
    // g is protected
    // x is not accessible from Derived_2
};

class Derived_3 : private Base {
    // f is private
    // g is private
    // x is not accessible from Derived_3
};
```

Public Inheritance Example

```
1   class Base {
2   public:
3       void func_1();
4   protected:
5       void func_2();
6   private:
7       int x_;
8   };
9
10  class Derived : public Base {
11  public:
12      void func_3() {
13          func_1(); // OK
14          func_2(); // OK
15          x_ = 0; // ERROR: inaccessible
16      }
17  };
18
19  struct Widget : public Derived {
20      void func_4() { func_2(); } // OK
21  };
22
23  int main() {
24      Derived d;
25      d.func_1(); // OK
26      d.func_2(); // ERROR: inaccessible
27      d.x_ = 0; // ERROR: inaccessible
28  }
```

Public Inheritance

```
#include <iostream>
using namespace std;

class Living{
    private:
    bool canBreadth=true;

    protected:
    bool canEat=true;

    public:
    bool immortal=false;
    void dis1(){
        cout<<canBreadth<<endl;
        cout<<canEat<<endl;
        cout<<immortal<<endl;
    }
};
```

```
class Dog:public Living{
    private:
    bool canBark=true;
    public:
    void dis2(){
        dis1();
    }

};

int main()
{
    Dog d;
    cout<<d.immortal<<endl;
    //cout<<d.canBreadth<<endl; Error
    //cout<<d.canEat<<endl; Error
    d.dis2();

    return 0;
}
```

Protected Inheritance Example

```
class Base {
public:
    void func_1();
protected:
    void func_2();
private:
    int x_;
};

class Derived : protected Base {
public:
    void func_3() {
        func_1(); // OK
        func_2(); // OK
        x_ = 0; // ERROR: inaccessible
    }
};

struct Widget : public Derived {
    void func_4() { func_2(); } // OK
};

int main() {
    Derived d; // OK: defaulted constructor is public
    d.func_1(); // ERROR: inaccessible
    d.func_2(); // ERROR: inaccessible
    d.x_ = 0; // ERROR: inaccessible
}
```

Protected Inheritance

```
#include <iostream>
using namespace std;

class Living{
    private:
    bool canBreadth=true;

    protected:
    bool canEat=true;

    public:
    bool immortal=false;
    void dis1(){
        cout<<canBreadth<<endl;
        cout<<canEat<<endl;
        cout<<immortal<<endl;
    }
};
```

```
class Dog:protected Living{
    private:
    bool canBark=true;
    public:
    void dis2(){
        dis1();
    }

};

int main()
{
    Dog d;
    // cout<<d.immortal<<endl; Error
    //cout<<d.canBreadth<<endl; Error
    //cout<<d.canEat<<endl; Error
    d.dis2();

    return 0;
}
```

Private Inheritance Example

```
class Base {
public:
    void func_1();
protected:
    void func_2();
private:
    int x_;
};

class Derived : private Base {
public:
    void func_3() {
        func_1(); // OK
        func_2(); // OK
        x_ = 0; // ERROR: inaccessible
    }
};

struct Widget : public Derived {
    void func_4() { func_2(); } // ERROR: inaccessible
};

int main() {
    Derived d; // OK: defaulted constructor is public
    d.func_1(); // ERROR: inaccessible
    d.func_2(); // ERROR: inaccessible
    d.x_ = 0; // ERROR: inaccessible
}
```

Private Inheritance

```
#include <iostream>
using namespace std;

class Living{
    private:
    bool canBreadth=true;

    protected:
    bool canEat=true;

    public:
    bool immortal=false;
    void dis1(){
        cout<<canBreadth<<endl;
        cout<<canEat<<endl;
        cout<<immortal<<endl;
    }
};
```

```
class Dog:Living{
    private:
    bool canBark=true;
    public:
    void dis2(){
        dis1();
    }

};

int main()
{
    Dog d;
    //cout<<d.immortal<<endl; Error
    //d.dis1(); Error
    d.dis2();

    return 0;
}
```

Public Inheritance

- public inheritance is inheritance in traditional object-oriented programming sense
- public inheritance models an ***is-a*** relationship (i.e., derived class object is a base class object)
- most common form of inheritance
- inheritance relationship visible to all code

Protected and Private Inheritance

- protected and private inheritance not inheritance in traditional object-oriented programming sense (i.e., no is-a relationship)
- form of implementation inheritance
- ***implemented-in-terms-of*** relationship (i.e., derived class object implemented in terms of a base class object)
- in case of protected inheritance, inheritance relationship only seen by derived classes and their friends and class itself and its friends
- in case of private inheritance, inheritance relationship only seen by class itself and its friends (not derived classes and their friends)
- except in special circumstances, normally bad idea to use inheritance for composition
- one good use case for private/protected inheritance is in policy-based design, which exploits empty base optimization (EBO)

Constructors and Destructors in Inheritance

- Single and multilevel inheritance – base class constructors are called first, then derived class constructors are called
  - E.g. class B : public A
  - Constructor of A is called first, then of B.

- Multiple inheritance – base class constructors are called from left to right as specified in derived class inheritance list. Then derived class constructors are called.
  - E.g. class C : public A, public B
  - Here constructor of A is called first, then constructor of B is called and then of derived class C

- Destructors are called in the reverse order of constructors

Order of Construction

```
1  #include <vector>
2  #include <string>
3
4  class Base {
5  public:
6      Base(int n) : v_(n, 0) {}
7      // ...
8  private:
9      std::vector<char> v_;
10 };
11
12 class Derived : public Base {
13 public:
14     Derived(const std::string& s) : Base(1024), s_(s)
15       { i_ = 0; }
16     // ...
17 private:
18     std::string s_;
19     int i_;
20 };
21
22 int main() {
23     Derived d("hello");
24 }
```

- construction order for `Derived` constructor: 1) `Base` class object, 2) data member `s_`, 3) `Derived` constructor body (initializes data member `i_`)

Constructor and Destructor

```
#include<iostream>
using namespace std;
class Teacher{
    string course;
    public:
    Teacher(string course){
        this->course=course;
        cout<<"Parent Constructor"<<endl;}
    ~Teacher(){
        cout<<"Parent Destructor"<<endl; }
};
class Principal:public Teacher{
    string _name;
    public:
    Principal(string course,string name):Teacher(course),_name(name){
        cout<<"Child Constructor"<<endl; }
    ~Principal(){
        cout<<"Child Destructor"<<endl;}
};
int main(){
    Principal principal("OOP","Dan");
    return 0;}
```

Function Overriding (I)

- Functions with same name and same parameters and same return type
- Defined in base class and derived classes
- When derived class object calls the function, it calls overridden function in the derived class
- When base class object calls the function, it calls the base class copy of the function

Function Overriding (II)

- Example of function overriding

```
class Base
{
public:
void display()
{ cout<<"I am base"<<endl;}
};

class Derived : public Base
{
public:
void Display()                    //function overriding
{ cout<<"I am derived"<<endl;}
};

void main()
{
Base b;
b.display(); //base class definition called
             //o/p- I am base
Derived d;   //derived class definition called
d.display(); //o/p- I am derived

}
```

Method overridden

```
#include<iostream>

using namespace std;
class Staff{
    public:
    void display(){
        cout<<"From staff with number: "<<endl;
    }
};
class Teacher: public Staff{
    public:
    void display(){
        cout<<"From Teacher"<<endl;
        Staff::display();
    }
};
int main(){
    Teacher teacher;
    teacher.Staff::display(); // display is overriden
    teacher.display();
    return 0;
}
```

Override Control: The `override` Qualifier

- when looking at code for derived class, often not possible to determine if member function intended to override virtual function in base class (or one of its base classes)
- can sometimes lead to bugs where programmer expects member function to override virtual function when function not virtual
- **`override`** qualifier used to indicate that member function is expected to override virtual function in parent class; must come at end of function declaration
- example:

```
class Person {
public:
    virtual void print() const;
    // ...
};

class Employee : public Person {
public:
    void print() const override; // must be virtual
    // ...
};
```

Override Control: The `final` Qualifier

- sometimes, may want to prevent any further overriding of virtual function in any subsequent derived classes
- adding **`final`** qualifier to declaration of virtual function prevents function from being overridden in any subsequent derived classes
- preventing further overriding can sometimes allow for better optimization by compiler (e.g., via devirtualization)
- example:

```
class A {
public:
    virtual void doStuff();
    // ...
};

class B : public A {
public:
    void doStuff() final; // prevent further overriding
    // ...
};

class C : public B {
public:
    void doStuff(); // ERROR: cannot override
    // ...
};
```

Hiding Base-Class Member Functions in Derived Class

- can provide new versions of member functions in derived class to hide original functions in base class

```
1  #include <iostream>
2
3  class Fruit {
4  public:
5      void print() const {std::cout << "fruit\n";}
6  };
7
8  class Apple : public Fruit {
9  public:
10     void print() const {std::cout << "apple\n";}
11 };
12
13 class Banana : public Fruit {
14 public:
15     void print() const {std::cout << "banana\n";}
16 };
17
18 int main() {
19     Fruit f;
20     Apple a;
21     Banana b;
22     f.print(); // calls Fruit::print
23     a.print(); // calls Apple::print
24     b.print(); // calls Banana::print
25 }
```

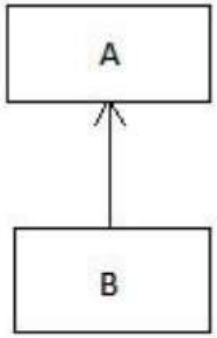
Inheritance (IV)

- Class can inherit properties of one or more classes or from more than one level.
- Depending on the number of base classes and number of levels, 5 Types of inheritance:
  - Single inheritance
  - Multilevel inheritance
  - Multiple inheritance
  - Hybrid inheritance
  - Hierarchical inheritance

Single Inheritance

- Derived class has only one base class
- All properties of base class are available in derived class. But vice versa not true
- Object of derived class can access all public properties of base class

```
class A
{...
};

class B : public A
{...
};
```



Single Inheritance

```
#include<iostream>
using namespace std;
class Shape{
    string color="white";
    public:
    void display(){
        cout<<color<<endl;}
};
class Triangle:public Shape{
    string color="Red";
    public:
    void display(){
        cout<<color<<endl;
        Shape::display();}
};
```

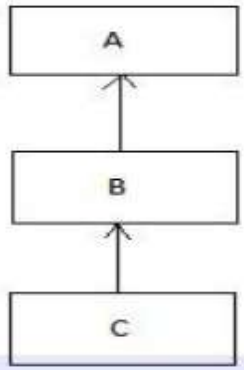
Single Inheritance

```
int main(){
    Triangle triangle;
    triangle.display();

    Shape shape;
    shape.display();
    return 0;
}
```

Multilevel Inheritance

- Derived class becomes base class to another class

```
class A
{...
};

class B : public A
{...
};

class C : public B
{...
};
```



- Here B is called intermediate base class
- All the public properties of A are available in C
- Private properties of A not accessible in C

Multilevel Inheritance

```
#include<iostream>
using namespace std;
class Polygon{
    public:
    void display(){
       cout<<”Polygon”<<endl;}
};
class Quadrilateral: public Polygon{
    public:
    void display(){
cout<<”Quadrilateral”<<endl;
};
```

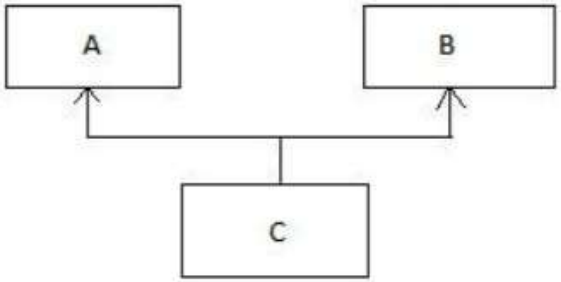
Multilevel Inheritance

```
class Rectangle:public Quadrilateral{
    public:
    void display(){
        cout<<"Rectangle”<<endl;
    }
};
int main(){
    Rectangle rectangle;
    rectangle.display();
    return 0;
}
```

Multiple Inheritance

- Derived class has more than one base class
- Derived class has all the public and protected properties of all the base classes
- Each base class can be inherited with any visibility mode. All are separated by a comma

```
class A
{...
};

class B
{...
};

class C : public A, public B
{...
};
```



Multiple Inheritance

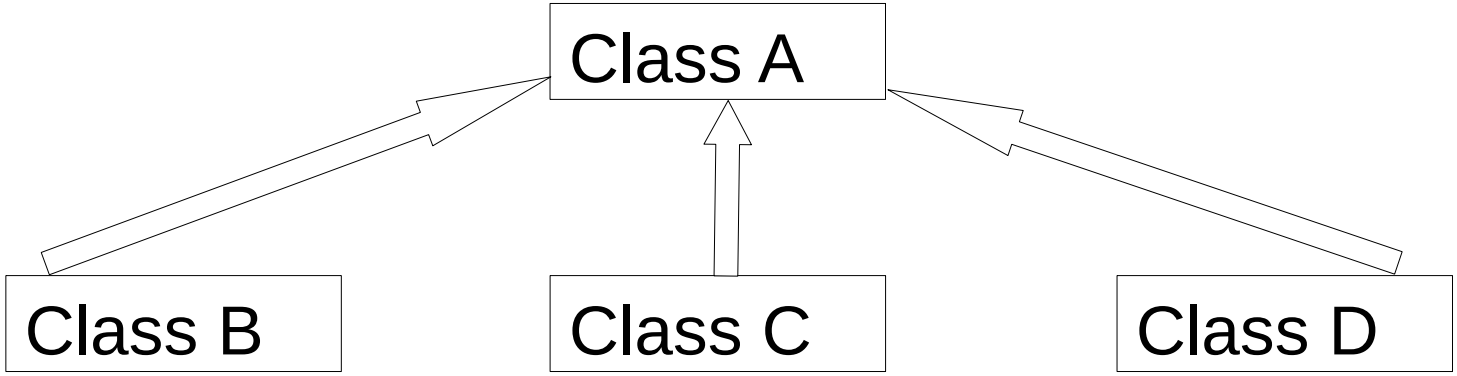
```
#include<iostream>
using namespace std;
class Staff{
    public:
    void display(){
        cout<<"Staff"<<endl;}
};
class Teacher{
    void display(){
        cout<<"Teacher"<<endl;}
};
```

Multiple Inheritance

```
class Principal:public Teacher,public Staff{
    public:
    Principal(string course,string name):Teacher(course),Staff(name)
     {    }
};
int main(){
    Principal principal("OOP","Dan");
    principal.Teacher::display(); // display is ambiguous
    return 0;
}
```

Hierarchical Inheritance

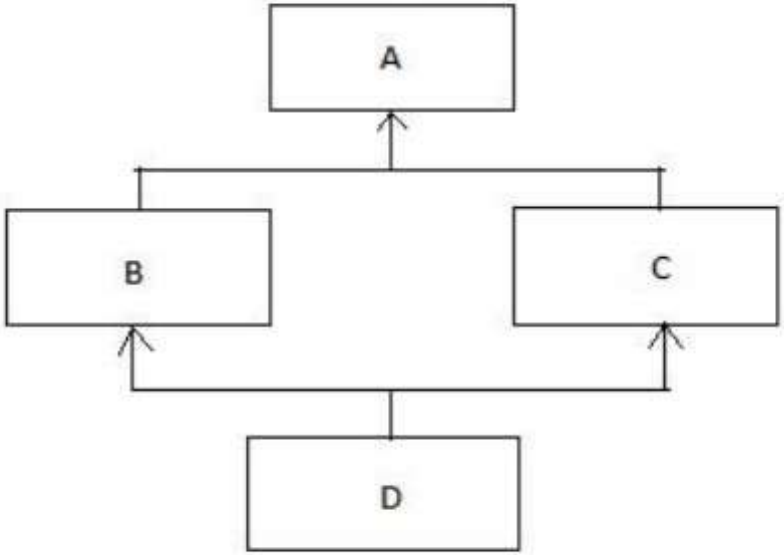
Multiple classes are inherited from single class



Hybrid Inheritance

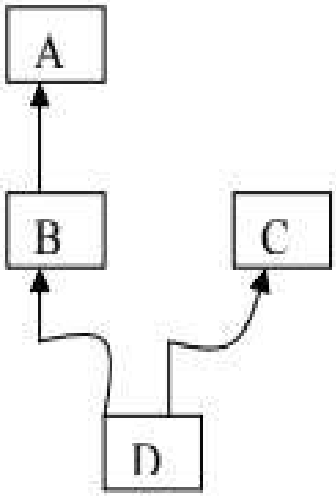
- Derived class has multiple base classes
- These intermediate base classes have a common base class
- To avoid getting multiple copies of common base class in the derived class, intermediate base classes inherit the base class as virtual
- Hence only one copy of base class will be given in derived class

```
class A
{...
};

class B : virtual public A
{...
};

class C : virtual public A
{...
};

class D : public B, public C
{...
};
```

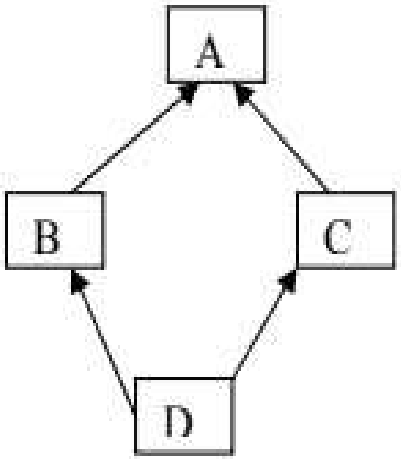


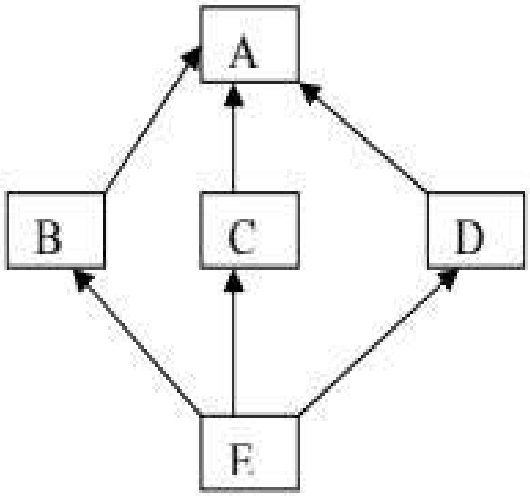
Hybrid Inheritance

Involves more than one type of inheritance



Hybrid (multilevel + multiple)





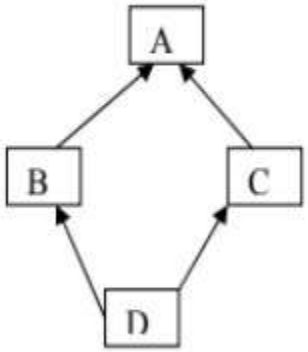
[multilevel , Hierarchical , multiple]
→ Hybrid

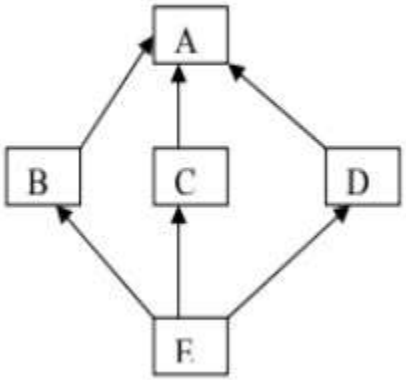
Hybrid Inheritance

```
#include<iostream>
using namespace std;
class A{
    public:
    void display(){
        cout<<"From A: "<<endl;
    }
};
class B: public A{
};

class C: public A{
};

class D: public C, B{
};
int main(){
    D obj;
    //obj.B::A::display(); Error
    return 0;
}
```

Multipath Inheritance

- If a class is inherited from multiple paths, then it is called multipath inheritance
- It will surely have ambiguity
- Ambiguity can be solved only using virtual functions.





Multipath Inheritance

```
#include<iostream>
using namespace std;
class A{
    public:
    void display(){
        cout<<"From A: "<<endl;
    }
};
class B: public A{
};

class C: public A{
};

class D: public C, B{
};
int main(){
    D obj;
    //obj.B::A::display(); Error
    return 0;
}
```

Association

The association is the relationship between two objects.
The association is used for communication between the objects.
Association will exists when two objects interact with each other.
For e.g student registers for a course.
Customer opens bank account.
Person click photo with camera.

Aggregation

Aggregation is has relationship.
It is weak type of relationship between the objects.
There may not be physical containment.
Life of one object does not depend upon the life of another object.
It is shown using unfilled diamond.

Aggregation

```
class Car{
        string color;
        int maxSpeed;
        Passenger *p;
        public:
        Car(){
            cout<<"Enter car's color and maxSpeed"<<endl;
            cin>>color>>maxSpeed;
        }
        void pickPassenger(){
            p=new Passenger;
        }
        void display(){
            cout<<"Color= "<<color<<endl;
            cout<<"MaxSpee= "<<maxSpeed<<endl;
            cout<<"Passenger name= "<<p->getName()<<endl;
}};
```

Aggregation

```
int main(){
    Car car;
    car.pickPassenger();
    car.display();
    return 0;
}
```

Aggregation

```
class Passenger{
    string name;
    public:
    Passenger(){
        cout<<"Enter passenger name"<<endl;
        cin>>name;
    }
    string getName(){
        return name;
    }
};
```

Composition

```
class Engine{
    string engineType;
    public:
    Engine(){
        cout<<"Enter engine type"<<endl;
        cin>>engineType;
    }
    string getEngineType(){
        return engineType;
    }
};
```

Composition

```
class Car{
        string color;
        int maxSpeed;
        Engine e;
        public:
        Car(){
            cout<<"Enter car's color and maxSpeed"<<endl;
            cin>>color>>maxSpeed;
        }
        void display(){
            cout<<"Color= "<<color<<endl;
            cout<<"MaxSpee= "<<maxSpeed<<endl;
            cout<<"Engine type= "<<e.getEngineType()<<endl;
        }
};
```

Composition

```
int main(){
    Car car;
    car.display();
    return 0;
}
```